

Thread Pool in C# with Examples

In this article, I am going to discuss **Thread Pool in C# with Examples**. Please read our previous article where we discussed the [Performance Testing of a Multithreaded Application](#) in C#. As part of this article, we are going to discuss the following pointers.

1. [The Request Life cycle of a Thread.](#)
2. [What is Thread Pooling in C#?](#)
3. [Why do we need a C# Thread Pool?](#)
4. [Performance testing between normal thread and thread pooling](#)

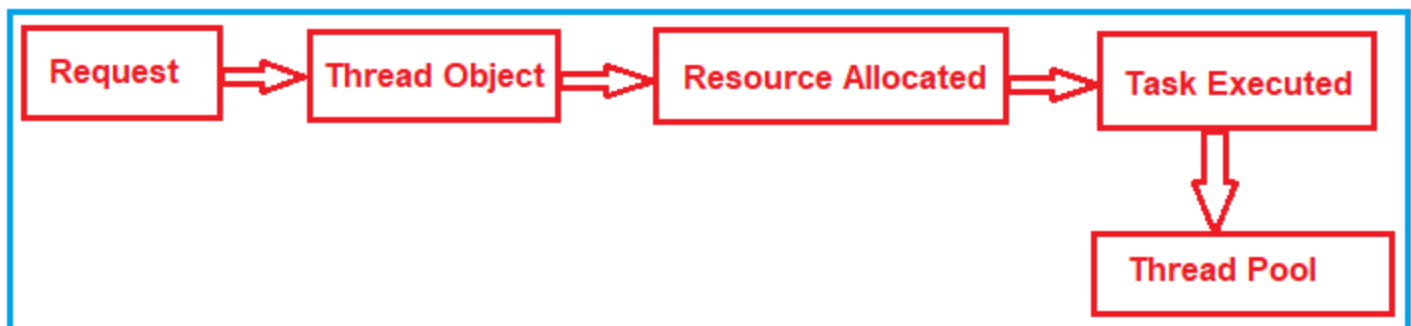
The Request Life cycle of a Thread in C# with Example.

Let us understand the life cycle of a thread in C#. In order to understand this, please have a look at the following image. When the .NET framework receives a request (the request can be a method call or function call from any kind of application). To handle that request, a thread object is created. When the thread object is created some resources are allocated to that thread object such as memory. After then the task is executed and once the task is completed then the garbage collector removes that thread object to free up memory allocation. This is the life cycle of a thread in C#.



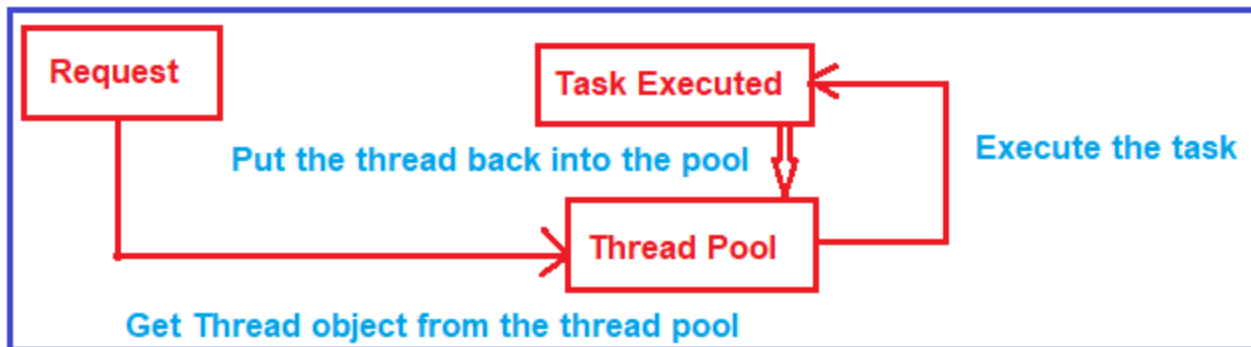
These steps are going to be repeated again and again for each request that comes in a multithread application. That means every time a new thread object is created, it gets allocated in the memory. If there are many requests then there will be many thread objects and if there are many thread objects then there will be a load on the memory which slows down your application.

There is great room for performance improvements. The Thread object is created, resources are allocated, the task is executed, and then it should not go for garbage collection, instead of how about taking the thread object and putting it into a pool as shown in the below image? This is where thread pooling comes into the picture.



Thread Pool in C#:

The Thread pool in C# is nothing but a collection of threads that can be reused to perform a number of tasks in the background. Now when a request comes, then it directly goes to the thread pool and checks whether there are any free threads available or not. If available, then it takes the thread object from the thread pool and executes the task as shown in the below image.



Once the thread completes its task then it is again sent back to the thread pool so that it can reuse. This reusability avoids an application to create a number of threads and this enables less memory consumption.

How to use Thread Pool in C#?

Let us see a simple example to understand how to use Thread Pooling in C#. Once you understand how to use thread pooling, then we will see the performance benchmark between the normal thread object and the thread pool thread object.

Step1:

In order to implement thread pooling in C#, first, we need to import the Threading namespace as `ThreadPool` class belongs to this Threading namespace as shown below.

```
using System.Threading;
```

Step2:

Once you import the Threading namespace, then you need to use the **ThreadPool** class, and using this class you need to call the **QueueUserWorkItem** static method. And, if you go to the definition of the **QueueUserWorkItem** method, then you will see that this method takes one parameter of the type **WaitCallback** object. While creating the object of the **WaitCallback** class, you need to pass the method name that you want to execute as shown below.

```
ThreadPool.QueueUserWorkItem(new WaitCallback(MyMethod));
```

Here, the **QueueUserWorkItem** method Queues the function for execution and that function executes when a thread becomes available from the thread pool. If no thread is available then it will wait until one thread gets freed. Here `MyMethod` is the method that we want to execute by a thread pool thread.

Complete Example Code to Understand Thread Pooling in C#

As you can see in the below code, here, we create one method that is MyMethod and as part of that method, we simply print the thread id, whether the thread is a background thread or not, and whether it is from a thread pool or not. And we want to execute this method 10 times using the thread pool threads. So, here we use a simple for each loop and use the ThreadPool class and call that method.

```
using System;
using System.Threading;
namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 10; i++)
            {
                ThreadPool.QueueUserWorkItem(new WaitCallback(MyMethod));
            }
            Console.Read();
        }
        public static void MyMethod(object obj)
        {
            Thread thread = Thread.CurrentThread;
            string message = $"Background: {thread.IsBackground}, Thread Pool: {thread.IsThreadPoolThread}, Thread ID: {thread.ManagedThreadId}";
            Console.WriteLine(message);
        }
    }
}
```

Once you execute the above code, it will give you the following output. As you can see, it shows that it is background threads and the threads are from the thread pool and the thread ids may vary in your output. Here, you can see some of the threads are reused and this is the advantage of using Thread Pooling in C#.

```
Background: True, Thread Pool: True, Thread ID: 3
Background: True, Thread Pool: True, Thread ID: 3
Background: True, Thread Pool: True, Thread ID: 9
Background: True, Thread Pool: True, Thread ID: 6
Background: True, Thread Pool: True, Thread ID: 4
Background: True, Thread Pool: True, Thread ID: 7
Background: True, Thread Pool: True, Thread ID: 9
Background: True, Thread Pool: True, Thread ID: 6
Background: True, Thread Pool: True, Thread ID: 5
Background: True, Thread Pool: True, Thread ID: 3
```

Now, let us see the same example using the normal threads. Please modify the code as follows. Here, instead of using ThreadPool, we are using the Thread class to create the threads and invoke the methods.

```
using System;
using System.Threading;
```

```

namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 10; i++)
            {
                Thread thread = new Thread(MyMethod)
                {
                    Name = "Thread" + i
                };
                thread.Start();
            }
            Console.Read();
        }
        public static void MyMethod(object obj)
        {
            Thread thread = Thread.CurrentThread;
            string message = $"Background: {thread.IsBackground}, Thread Pool:
{thread.IsThreadPoolThread}, Thread ID: {thread.ManagedThreadId}";
            Console.WriteLine(message);
        }
    }
}

```

Now, run the above code, and you will get the following output. Here, you can see, the threads are not backgrounded threads and they are also not coming from the thread pool, and the threads are also not reused. Each thread has a unique thread ID.

```

Background: False, Thread Pool: False, Thread ID: 3
Background: False, Thread Pool: False, Thread ID: 7
Background: False, Thread Pool: False, Thread ID: 6
Background: False, Thread Pool: False, Thread ID: 4
Background: False, Thread Pool: False, Thread ID: 5
Background: False, Thread Pool: False, Thread ID: 8
Background: False, Thread Pool: False, Thread ID: 10
Background: False, Thread Pool: False, Thread ID: 9
Background: False, Thread Pool: False, Thread ID: 11
Background: False, Thread Pool: False, Thread ID: 12

```

Performance testing using and without using Thread Pool in C# with Example:

Let us see an example to understand the performance benchmark. Here, we will compare how much time the thread object takes and how much time the thread pool thread takes to do the same task i.e. to execute the same methods.

In order to do this, what we are going to do is, we will create a method called **Test** as shown below. This method takes an input parameter of the type object and as part of that Test method, we are doing nothing that means an empty method.

```
public static void Test(object obj)
{
}
```

Then we will create two methods such as **MethodWithThread** and **MethodWithThreadPool** and inside these two methods, we will create one for loop which will execute 10 times. Within for loop, we are going to call the Test as shown below. As you can see, the **MethodWithThread** method uses the Thread object to call the Test method while the **MethodWithThreadPool** method uses the ThreadPool object to call the Test method.

```
public static void MethodWithThread(.)
{
    for (int i = 0; i < 10; i++)
    {
        Thread thread = new Thread(Test);
    }
}
```

Uses Thread object to call the Test method

```
public static void MethodWithThreadPool( )
{
    for (int i = 0; i < 10; i++)
    {
        ThreadPool.QueueUserWorkItem(new WaitCallback(Test));
    }
}
```

Uses ThreadPool thread to call the Test method

Now we need to call the above two methods (**MethodWithThread** and **MethodWithThreadPool**) from the main method. As we are going to test the performance benchmark, we are going to call these two methods between the stopwatch start and end as shown below. The Stopwatch class is available in **System.Diagnostics** namespace. The for loop within the Main method is for warm-up. This is because when we run the code for the first time, compilation happens and compilation takes some time and we don't want to measure that.

```

for (int i = 0; i < 10; i++)
{
    MethodWithThread();
    MethodWithThreadPool();
}

```

This is warm up.

```

Stopwatch stopwatch = new Stopwatch();

Console.WriteLine("Execution using Thread");
stopwatch.Start(); //Start the stopwatch
MethodWithThread(); //call the method
stopwatch.Stop(); //Stop the stopwatch
Console.WriteLine("Time consumed by MethodWithThread is : " +
    stopwatch.ElapsedTicks.ToString());

stopwatch.Reset();

Console.WriteLine("Execution using Thread Pool");
stopwatch.Start(); //Start the stopwatch
MethodWithThreadPool(); //call the method
stopwatch.Stop(); //Stop the stopwatch
Console.WriteLine("Time consumed by MethodWithThreadPool is : " +
    stopwatch.ElapsedTicks.ToString());

```

The complete code is given below.

```

using System;
using System.Diagnostics;
using System.Threading;
namespace ThreadPoolApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            //Warmup Code start
            for (int i = 0; i < 10; i++)
            {
                MethodWithThread();
                MethodWithThreadPool();
            }
            //Warmup Code start
            Stopwatch stopwatch = new Stopwatch();
            Console.WriteLine("Execution using Thread");

```

```

        stopwatch.Start();
        MethodWithThread();
        stopwatch.Stop();
        Console.WriteLine("Time consumed by MethodWithThread is : " +
                           stopwatch.ElapsedTicks.ToString());

        stopwatch.Reset();
        Console.WriteLine("Execution using Thread Pool");
        stopwatch.Start();
        MethodWithThreadPool();
        stopwatch.Stop();
        Console.WriteLine("Time consumed by MethodWithThreadPool is : " +
                           stopwatch.ElapsedTicks.ToString());

        Console.Read();
    }
    public static void MethodWithThread()
    {
        for (int i = 0; i < 10; i++)
        {
            Thread thread = new Thread(Test);
            thread.Start();
        }
    }
    public static void MethodWithThreadPool()
    {
        for (int i = 0; i < 10; i++)
        {
            ThreadPool.QueueUserWorkItem(new WaitCallback(Test));
        }
    }
    public static void Test(object obj)
    {
    }
}
}

```

Output:

```

Execution using Thread
Time consumed by MethodWithThread is : 272588
Execution using Thread Pool
Time consumed by MethodWithThreadPool is : 692

```

As you can see in the above output, the Time consumed by MethodWithThread is 272588 and the Time consumed by MethodWithThreadPool is 692. If you observe there is a vast time difference between these two.

So it proves that the thread pool gives better performance as compared to the thread class object. If there are need to create one or two threads then you need to use the Thread class object while if there is a need to create more than 5 threads then you need to go for the thread pool class in a multithreaded environment.

That's it for today. In the next article, I am going to discuss [Foreground and Background Threads in C#](#) with examples. Here, in this article, I try to explain **Thread Pool in C#** with examples. I hope you enjoy this article and understood C# Thread Pooling with Examples.